HTTP Requests and Responses

1. Introduction

- HTTP (HyperText Transfer Protocol) operates on a **request–response model** between a client and a server.
- Every communication cycle involves a **client sending a request** and the **server returning a response**.
- This process forms the foundation of web communication — from loading a webpage to sending API data.

2. HTTP Request Overview

- A request is sent from the **client** (usually a browser or app) to the **server** to perform an action.
- It contains all the information the server needs to process the client's demand.
- Common actions include retrieving pages, submitting forms, and uploading data.

3. Structure of an HTTP Request

Each HTTP request consists of three key parts:

1. **Request Line:** Specifies the request method, resource (URL), and HTTP version.
   Example:
   `GET /index.html HTTP/1.1`
   - Components:
     - **Method:** Defines the action (e.g., GET, POST).
     - **URL/Path:** The location of the resource on the server.
     - **HTTP Version:** Defines the protocol version being used.
2. **Request Headers:** Provide **metadata** about the request.
   - Examples:
     - `Host: www.example.com`
     - `User-Agent: Mozilla/5.0`
     - `Accept: text/html`
     - `Content-Type: application/json`
   - Headers tell the server what kind of response the client expects and provide context about the client.
3. **Request Body (Optional):** Contains data being sent to the server, used mainly with **POST**, **PUT**, or **PATCH** methods.

Example (JSON):

```
{
 "username": "Aayush",
 "password": "12345"
        }
```

4. Common HTTP Methods

Method	Purpose	Typical Use
GET	Retrieve data from the server	Loading webpages, fetching data
POST	Send data to the server	Submitting forms, API requests
PUT	Update existing resources	Modifying user profiles or data
DELETE	Remove a resource	Deleting records or files
HEAD	Same as GET but retrieves only headers	Checking resource availability
OPTIONS	Shows supported methods	Debugging, CORS preflight requests

5. Example of a Complete HTTP Request

```
POST /login HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 27
username=Aayush&password=12345
```

- **Explanation:**
  - The method used is POST.
  - The resource is /login.
  - The body contains form data.
  - Headers describe the content and request context.

6. HTTP Response Overview

- Once the server processes the request, it sends back an **HTTP Response**.
- The response tells the client whether the request was successful and returns data or an error message.
- Like requests, responses have a **structured format**.

7. Structure of an HTTP Response

Status Line: Includes the HTTP version, status code, and status message.

Example:
```
HTTP/1.1 200 OK
```

- Components:
  - **Version:** HTTP/1.1 or HTTP/2.
  - **Status Code:** Numeric indicator of the result.
  - **Status Message:** Human-readable explanation.

2. **Response Headers:** Contain metadata about the response or server.
   - Examples:
     - Content-Type: text/html
     - Content-Length: 1024
     - Server: Apache/2.4.1
     - Set-Cookie: sessionId=abcd1234
3. **Response Body:** Contains the requested data (HTML, JSON, images, etc.).

Example:
```
<html>
 <body>
  <h1>Welcome, Aayush!</h1>
 </body>
</html>
```

8. Common HTTP Response Codes

Category	Code	Meaning
1xx – Informational	100 Continue	Request received, waiting for further steps
2xx – Success	200 OK, 201 Created	Request successfully processed
3xx – Redirection	301 Moved Permanently, 302 Found	Resource moved to another URL
4xx – Client Error	400 Bad Request, 401 Unauthorized, 404 Not Found	Client-side errors
5xx – Server Error	500 Internal Server Error, 503 Service Unavailable	Server failed to process request

9. Example of an HTTP Response

```
HTTP/1.1 200 OK
Date: Wed, 15 Oct 2025 10:00:00 GMT
Server: Apache/2.4.1
Content-Type: text/html
Content-Length: 1024
<html>
  <body>
    <h1>Welcome to Example.com!</h1>
  </body>
</html>
```

- **Explanation:**
  - Status code 200 OK indicates success.
  - Headers specify type and size of content.
  - Body contains HTML page data.

10. Request and Response Flow

Step-by-Step Communication:

1. The **client** sends a request to the server (e.g., GET /index.html).
2. The **server** receives and interprets the request.
3. The **server** processes it, retrieves or generates data.
4. The **server** sends back a structured **response**.
5. The **client** renders the data (in browsers or apps).

11. Persistent and Pipelined Requests

- **Persistent Connections:**
  - HTTP/1.1 introduced persistent connections using the Connection: keep-alive header.
    Enables multiple requests/responses over a single TCP connection.
- **Pipelining:**
  - Allows multiple HTTP requests to be sent before receiving responses.
  - Increases efficiency but rarely used due to head-of-line blocking.

12. Modern Enhancements (HTTP/2 and HTTP/3)

- **HTTP/2:**
  - Supports multiplexing (multiple requests at once).
  - Uses binary framing for efficiency.
- **HTTP/3:**
  - Runs over **QUIC (UDP)** instead of TCP.
  - Reduces latency and improves reliability.

Summary

- HTTP communication follows a **request–response model**.
- Requests contain **method, headers, and optional body**, while responses include **status, headers, and content**.
- Understanding this flow is essential for **web developers, network engineers, and API designers**.
- Modern HTTP versions improve performance, security, and reliability.